GLYVEX

Touchpad Gesture Recognition and Command Execution Using Java and Machine Learning

Final Semester Project — Technical Research-Style Project Report

Computer Science and Engineering

Registration No.	Team Member	Email
AP25110010712	Ashwath Raj Mulakalapalli	ashwath_m@srmap.edu.in
AP25110010120	BARATAM PRANEETH GUPTA	praneeth_baratam@srmap.edu.in
AP25110010905	Harsha Vardhan Kumar	harsha_jogi@srmap.edu.in
AP25110010841	Sai thanush reddy	thanush_bobbili@srmap.edu.in

Abstract

GLYVEX proposes a touchpad-centred interaction system in which a laptop touchpad is temporarily transformed from a conventional pointing surface into a gesture command interface. The interaction begins with a deliberate four-corner activation sequence intended to distinguish command input from ordinary touchpad use. Once activated, the user draws a predefined character or symbol. The resulting touch trajectory is captured as an ordered sequence of spatial and temporal samples, normalized and transformed into a representation suitable for machine-learning classification. The predicted gesture is subsequently resolved through a controlled command registry and can open a predefined desktop application or perform another explicitly configured action.

The project is designed as an integrated Java application rather than as an isolated machine-learning experiment. The architecture separates touch input, activation-state management, trajectory processing, machine-learning inference, command validation, user-interface configuration and testing. This separation is important because the practical difficulty of the system is distributed across the complete pipeline: a highly accurate classifier is of limited value if the touchpad cannot provide reliable input, while a reliable input layer is of limited value if uncertain predictions can trigger unintended commands.

The proposed implementation emphasizes a compact command vocabulary, reproducible dataset preparation, local inference, confidence-based rejection and allowlisted actions. The resulting system provides a concrete demonstration of human–computer interaction, gesture processing, machine learning and Java desktop engineering within one semester project.

Keywords: touchpad interaction, gesture recognition, machine learning, Java, trajectory classification, human–computer interaction, desktop automation, pattern recognition

1. Introduction

Human–computer interaction has traditionally been organized around a small set of highly visible input mechanisms: keyboards, pointing devices, touchscreens and graphical controls. These mechanisms are effective because their semantics are well established, but they also constrain the way commands are expressed. A keyboard shortcut requires memorizing a key combination; a graphical command requires locating a visible interface element; and a conventional touchpad gesture normally operates within a small set of gestures supported by the operating system or application.

GLYVEX explores an alternative interaction model in which a touchpad becomes a temporary command surface. The system does not attempt to replace normal pointing behavior. Instead, it introduces an activation mechanism that deliberately separates ordinary interaction from gesture-command interaction. The user first performs a four-corner sequence. If the sequence is valid, the system enters gesture mode and interprets subsequent movement as a symbol rather than as cursor control.

The central research and engineering question is therefore not simply whether a character can be classified. The more useful question is whether a complete desktop interaction pipeline can acquire a deliberately entered trajectory, recognize a compact gesture vocabulary and safely convert the result into an application action. This framing changes the project from a basic ML classifier into a small human–computer interaction system.

The design also creates a natural boundary for machine learning. Rather than attempting unrestricted handwriting recognition, the project can define a small number of gesture classes corresponding to useful actions. A symbol such as a circle, line, triangle or letter-like shape can become a semantic command. The classifier only needs to distinguish the supported vocabulary, while the command layer determines what each class means.

1.1 Project Motivation

A laptop touchpad is already physically available and requires no additional sensor hardware. It can capture a sequence of positional samples while remaining part of the normal computer interaction environment. Using it as a temporary gesture surface therefore provides an interesting platform for experimenting with implicit interfaces, custom gestures and ML-assisted interaction without requiring a dedicated touchscreen or external controller.

The project is also useful academically because it combines several subjects that are often implemented independently. Java provides the application and software architecture; machine learning provides pattern recognition; datasets provide empirical evidence; and operating-system input integration provides the connection to real hardware. The final system is therefore evaluated as an engineering pipeline rather than by the classifier alone.

1.2 Contributions of the Proposed System

The primary contribution is the definition and implementation of a complete interaction pipeline consisting of deliberate activation, trajectory capture, preprocessing, classification and controlled command execution. A secondary contribution is the use of a state machine to make the interaction deterministic and testable. A third contribution is the explicit separation between prediction and execution, which prevents a machine-learning output from directly becoming arbitrary executable input.

2. Problem Definition

The problem addressed by GLYVEX can be expressed as a sequence of transformations. Let a touchpad interaction produce a trajectory T containing n samples. Each sample consists of a two-dimensional position and a timestamp: T = {(x■,y■,t■), (x■,y■,t■), …, (x■,y■,t■)}. The system must first determine whether the trajectory belongs to a valid interaction session. If it does, the trajectory is transformed by a preprocessing function f(T) into a fixed or controlled representation X. A classifier g(X) then predicts one of the supported gesture classes.

The prediction is not the final output. Let C be the set of commands explicitly registered by the application. The predicted class k is mapped to a command only when k belongs to C and the prediction confidence satisfies the configured acceptance rule. Thus, the operational function can be represented as execute(h(g(f(T)))) where h is a validated command mapping rather than an arbitrary shell interpretation.

This distinction is important. A machine-learning classifier is inherently uncertain, particularly when the dataset is small or the classes are visually similar. Treating every prediction as an instruction would make recognition errors equivalent to command errors. GLYVEX therefore treats classification and execution as separate engineering stages.

2.1 Functional Requirements

- The application shall monitor the supported touchpad input source while remaining in an inactive command state.
- The application shall recognize a configured four-corner activation sequence within a specified time window.
- The application shall capture the trajectory produced after successful activation.
- The application shall preprocess trajectories using the same transformation during training and inference.
- The application shall classify the trajectory into the supported gesture vocabulary.
- The application shall reject predictions that fail the configured confidence or validity criteria.
- The application shall execute only commands present in an explicit allowlist.
- The application shall provide configuration and status information through a Java desktop interface.

2.2 Non-Functional Requirements

The system should provide low enough latency that recognition feels like part of the same interaction. It should remain deterministic at the state-machine level, fail safely when input is incomplete and provide sufficient logging to reproduce failures. The model should be small enough for local execution, and the software should be structured so that the touchpad adapter can be replaced without rewriting the ML or command layers.

3. Related Concepts and Design Context

GLYVEX draws on several established areas of computing rather than proposing a new machine-learning paradigm. Gesture recognition commonly represents an interaction as a sequence of spatial samples and attempts to determine which gesture class best explains the sequence. Classical approaches can use geometric features, directional changes, path length and temporal characteristics, while more complex approaches can learn directly from sequences.

Touch interfaces also distinguish between continuous pointing and intentional gesture input. A useful gesture system must decide when movement should be interpreted as a gesture and when it should remain ordinary pointer activity. GLYVEX addresses this segmentation problem explicitly through its four-corner activation mechanism.

The project also follows a common pattern in safe automation systems: a recognition layer proposes an interpretation, while a policy layer determines whether the interpretation is allowed to cause an action. This separation is especially appropriate for a system in which ML uncertainty exists by design.

3.1 Why a Compact Gesture Vocabulary

A compact vocabulary offers three advantages. First, the dataset requirement becomes manageable within a semester. Second, the class boundaries can be inspected directly through confusion matrices and example trajectories. Third, the command mapping remains understandable during a demonstration. A system with six clearly defined classes can be evaluated more rigorously than a system that claims to recognize unrestricted handwriting using a small collection of ad hoc examples.

3.2 Why Not Use a Permanent Drawing Window

A permanent drawing canvas would simplify input capture but would also change the nature of the project. The distinctive interaction is that the touchpad itself becomes the drawing surface after activation. A separate canvas could still be used as a development tool for dataset collection and replay, but the deployed interaction should preserve the touchpad-centric concept.

Design principle

The user should not need to navigate to a separate application before drawing the command. The four-corner sequence is the transition from ordinary touchpad behavior to temporary gesture interaction.

4. Overall System Architecture

The proposed architecture is divided into seven logical components: input acquisition, activation and state management, trajectory capture, preprocessing, machine-learning inference, command resolution and user-interface configuration. Each component has a narrow responsibility and communicates with adjacent components through explicit data structures.

4.1 Input Acquisition

The input layer is responsible for converting the physical touchpad events exposed by the target operating system into a common representation. The remainder of the application should not need to know whether the events came from a native Java API, a platform bridge or a helper process. A normalized TouchSample object can contain x and y coordinates, a timestamp and, if available, contact-state information.

4.2 Activation and State Management

The state manager receives touch samples and determines whether the application is monitoring, recognizing the corner sequence, capturing a gesture, processing the result or executing a command. This prevents the application from interpreting the same physical movement simultaneously as activation and drawing.

4.3 Processing and Inference

The processing layer receives a completed trajectory and applies the exact preprocessing pipeline used during model development. The ML layer then receives a feature representation and returns a predicted class together with an associated confidence or score. The inference layer should not launch applications itself.

4.4 Command Resolution

The command layer receives a semantic identifier such as OPEN_CALCULATOR or OPEN_EDITOR. It searches an allowlisted registry and resolves the identifier to a known Java command implementation. This architecture is intentionally more restrictive than directly executing text received from the classifier.

4.5 User Interface

The JavaFX interface provides configuration, status and diagnostics. It should expose the current interaction state, the available gesture mappings, relevant thresholds and the ability to disable gesture execution. It should not become a dependency of the core recognition pipeline.

5. Touchpad Input Acquisition

Touchpad input is the most hardware-dependent part of GLYVEX. A normal desktop Java event system may expose mouse movement and button events without exposing the complete underlying touch trajectory. Consequently, the project must verify the actual event source available on the selected laptop before assuming that a pure Java API can provide the required information.

The recommended software contract is an abstraction such as TouchInputSource with start and stop operations and a listener receiving TouchSample objects. The implementation beneath this interface may use a native operating-system facility or a small helper process. The architectural goal is to isolate this uncertainty to one module.

A sample should contain normalized coordinates where possible. If the physical touchpad has a known width and height, raw coordinates can be mapped to a $[0,1] \times [0,1]$ space. This makes the later stages independent of the absolute resolution reported by the device.

5.1 Event Integrity

Event loss can distort the geometry of a gesture. If movement samples are dropped, a smooth curve may appear as a set of large jumps. The capture layer should therefore preserve the timestamp and maintain the order in which events arrive. If the underlying interface provides contact identifiers, the system should use them to distinguish a real gesture from unrelated pointer activity.

5.2 Hardware Feasibility Test

Before model development begins, the team should create a small diagnostic program that displays the current touch coordinates and records raw events to a file. The diagnostic output should be inspected for coordinate stability, sampling frequency, movement continuity and correct start/end detection. This test is more valuable early in development than a polished graphical interface.

Critical feasibility gate

If the target laptop cannot expose usable touch coordinates through the selected Java-compatible mechanism, the project must adapt the input layer before the ML architecture is finalized. The classifier cannot solve a hardware acquisition problem.

6. Four-Corner Activation Mechanism

The four-corner activation sequence is the principal mechanism that separates deliberate commands from ordinary touchpad movement. The touchpad is divided into four conceptual corner regions. A tap is considered a corner activation when its coordinates fall within the tolerance region of the expected corner and its timing satisfies the sequence constraints.

A first implementation can use a fixed sequence such as top-left, top-right, bottom-right and bottom-left. The sequence should be configurable after the basic system is validated. Each accepted corner advances the activation state. An incorrect corner or a timeout resets the sequence.

6.1 Timing and Debouncing

Two timing parameters are important. The first is the maximum interval between successive corner taps. The second is a debounce interval that prevents multiple events generated by a single physical tap from being counted as multiple activations. These values should be configurable and experimentally tuned on the target device.

6.2 Activation Tolerance

A tolerance that is too small makes the sequence frustrating because the user must hit an exact coordinate. A tolerance that is too large makes accidental activation more likely. The appropriate value should therefore be established empirically using repeated trials rather than selected arbitrarily.

6.3 State Transition

Successful completion of the sequence does not execute a command. It only moves the state into gesture mode. This distinction is essential because the activation sequence itself is a control mechanism rather than a gesture class.

7. Gesture Capture and Trajectory Representation

After activation, the system begins recording movement samples. A trajectory is naturally represented as an ordered sequence of points with timestamps. The capture module should define exactly when recording starts and ends. Starting too early can include the final corner tap; ending too late can include subsequent ordinary touch activity.

A practical rule is to enter capture mode after the activation sequence is accepted and begin recording when movement exceeds a small displacement threshold. Capture ends on release, inactivity timeout or an explicit cancellation event. The trajectory should be rejected if it contains too few meaningful samples.

7.1 Geometric Features

The trajectory can provide several useful features without requiring a complex model. The bounding box describes the extent of the gesture. Path length captures the amount of movement. Direction changes describe local shape. The displacement between the first and last point provides a coarse measure of whether the gesture returns toward its origin. Curvature approximations can also distinguish rounded and angular symbols.

7.2 Temporal Features

Timestamps make it possible to compute instantaneous or segment-level speed. Timing can be useful when two gestures have similar geometry but different drawing patterns. However, timing can also introduce unwanted user-specific variation. The final feature set should therefore be chosen through experiments rather than assuming that every available signal is beneficial.

7.3 Variable-Length Input

Different users and drawing speeds produce different numbers of samples. A classifier expecting a fixed-length vector requires resampling or feature aggregation. Resampling the trajectory at evenly spaced path intervals is one practical approach because it preserves the overall geometry while reducing dependence on the original event sampling rate.

8. Preprocessing Pipeline

Preprocessing is the bridge between physical input and machine learning. The same mathematical transformation must be used during training and deployment. If training data is normalized differently from live trajectories, even a strong classifier can fail because its input distribution has changed.

8.1 Coordinate Normalization

Raw coordinates should first be transformed into a consistent coordinate system. If the touchpad reports values in device-specific units, dividing by the available width and height produces normalized coordinates. This prevents the model from learning the physical resolution of one device.

8.2 Translation and Scale Normalization

The absolute position of a symbol is usually irrelevant. Centering the trajectory relative to its centroid or bounding box removes this nuisance variation. Scaling the trajectory to a standard size reduces differences caused by the user drawing the same symbol at different magnitudes.

8.3 Resampling

A fixed number of points can be obtained by resampling according to cumulative path length. This method is preferable to selecting every kth raw event because event frequency may vary. The resulting points correspond to approximately equal portions of the drawn path.

8.4 Smoothing

Touch input can contain small fluctuations. A smoothing filter can reduce high-frequency noise, but excessive smoothing may remove meaningful corners. Smoothing should therefore be applied conservatively and validated against actual gesture samples.

8.5 Feature Vector

The final feature vector can combine normalized coordinates, direction changes, segment lengths and selected temporal features. Alternatively, the complete resampled sequence can be supplied to a sequence-aware model. The first implementation should favor the representation that can be measured, visualized and debugged easily.

9. Dataset Construction

The dataset should be treated as an engineering artifact rather than as an afterthought. A gesture classifier can appear highly accurate when tested on a small collection of nearly identical drawings, yet fail when a different user draws the same symbol at another position or speed.

For this reason, each class should contain multiple examples from multiple contributors. The dataset should record the class label and enough session metadata to distinguish independent collection sessions. The metadata need not contain personal information; a simple anonymous participant identifier is sufficient for evaluating user-independent generalization.

9.1 Class Selection

The initial vocabulary should contain a manageable number of visually distinguishable symbols. Each class should correspond to a meaningful action. For example, a circular gesture could map to a calculator, a square-like gesture to a text editor and another symbol to a browser. The exact vocabulary should be chosen by the team based on ease of drawing and command usefulness.

9.2 Data Diversity

Examples should vary in size, location, drawing speed and starting point. The goal is not to make the data noisy for its own sake, but to ensure that the classifier learns the symbol rather than a fixed physical position or drawing speed.

9.3 Training, Validation and Test Sets

The dataset should be divided before extensive model tuning. A stronger evaluation can reserve participants for testing so that the final test set contains users not seen during training. This provides evidence about whether the system generalizes beyond the team members who collected the training data.

Dataset principle

More data is not automatically better. Correct labels, meaningful variation and a clean evaluation split are more valuable than a large collection of nearly identical trajectories.

10. Machine-Learning Methodology

The classification problem can be formulated as supervised multiclass learning. Each preprocessed trajectory X is associated with a class label y. The model learns a function g(X) that estimates the class corresponding to the gesture. During deployment, the model receives a trajectory generated by a user and returns the most plausible class.

A classical baseline should be established before introducing a more complex model. k-nearest neighbors, a decision tree, random forest or support-vector machine can provide useful reference points. A small multilayer perceptron can then be evaluated if the feature representation requires nonlinear decision boundaries.

10.1 Baseline First

A baseline is important because it provides evidence about whether additional model complexity is actually helping. If a simple classifier performs well, the project gains a strong argument for keeping the deployed model compact. If the baseline performs poorly, the team can inspect whether the problem is data, preprocessing or model capacity before escalating complexity.

10.2 Model Selection Criteria

The final model should be evaluated using classification quality, inference latency, model size and stability. A slightly more accurate model may not be useful if it introduces unnecessary runtime complexity or requires a large external dependency.

10.3 Confidence and Rejection

The application should not assume that the highest-scoring class is always correct. A configurable acceptance threshold can cause the system to reject uncertain trajectories. Rejection is not a failure of the ML component; it is a necessary behavior when the cost of executing the wrong command is greater than the cost of asking the user to try again.

11. Evaluation Methodology

The evaluation should measure both recognition performance and interaction reliability. Classification metrics alone do not capture activation errors, capture failures or command latency. The project should therefore use a layered evaluation procedure.

11.1 Classification Metrics

Accuracy measures the fraction of correctly classified samples. Precision measures how often predictions of a class are correct, while recall measures how often samples belonging to that class are detected. The F1-score provides a combined view of precision and recall. A confusion matrix is particularly useful because it reveals which gesture pairs are difficult to distinguish.

11.2 Interaction Metrics

The team can measure four-corner activation success rate, average time to activation, gesture capture failure rate, recognition latency and command execution latency. These measurements describe whether the complete system behaves consistently rather than only whether the classifier is statistically accurate.

11.3 User-Independent Testing

When possible, the final test should contain trajectories from users not represented in training. This is a stronger test of gesture recognition because it reduces the possibility that the model has simply memorized an individual's drawing style.

11.4 Error Analysis

Every major error category should be inspected. If two symbols are confused, the team should compare their normalized trajectories. If errors occur mainly at unusual scales, the normalization stage may be incomplete. If errors occur for only one user, the issue may be user-specific drawing style. This analysis is more informative than reporting one accuracy number.

12. Java Application Architecture

Java is the primary implementation environment. The project should use a modular package structure so that hardware integration, gesture processing, ML inference and command execution remain separate. A typical package hierarchy can include app, input, gesture, processing, ml, commands, config, ui and test.

12.1 Core Interfaces

The input layer can expose an interface such as TouchInputSource. The gesture layer can expose a GestureSession or state-machine controller. The ML layer can expose a Predictor interface whose implementation loads the trained model. The command layer can expose Command objects with a controlled execute operation.

12.2 Configuration

Configuration should not be hard-coded into source files. Corner tolerance, timeouts, confidence thresholds, gesture labels and command mappings can be stored in a structured configuration file. The application should validate configuration at startup and reject invalid mappings rather than failing later during execution.

12.3 Logging

Logging is especially important for a project involving physical interaction because failures are difficult to reproduce from memory. The application should record state transitions, activation failures, capture duration, model predictions, confidence values and command outcomes. Raw touch trajectories can be stored separately when debugging is enabled.

12.4 Packaging

The final Java application can be packaged as a desktop executable or platform-specific application bundle using the Java packaging facilities permitted by the course environment. The packaged application should include the model artifact and configuration required for local inference.

13. Command Execution Layer

The command layer converts a semantic prediction into a controlled desktop action. The most important design rule is that the ML output must not be treated as executable text. The model should return an identifier such as OPEN_CALCULATOR, and a command registry should resolve that identifier to a known implementation.

For example, the registry can contain a CalculatorCommand whose execute method invokes the appropriate Java desktop mechanism. Another command can open a predefined directory. The mapping can be configured without allowing arbitrary command-line strings to be supplied by the model.

13.1 Allowlisting

Only commands registered in the allowlist should be executable. Unknown identifiers should be rejected. This is a simple but important boundary because an ML system can produce unexpected outputs under noisy input.

13.2 Failure Handling

If an application cannot be opened, the command layer should report the failure and return the state machine to a safe state. The application should not remain in an execution state indefinitely. A command failure should also not cause the input monitor to terminate.

13.3 Safe Demonstration Commands

The final demonstration can use benign actions such as opening a calculator, text editor, browser or a predefined folder. These actions are sufficient to demonstrate that the recognition result has been propagated through the complete application without introducing unnecessary system-level risk.

Separation of concerns

Recognition answers “What gesture was drawn?” The command layer answers “What is this gesture allowed to do?” Keeping these questions separate makes the system easier to test and safer to operate.

14. User Interface Design

The permanent graphical interface should be deliberately small. The central interaction occurs on the touchpad, so the UI should primarily provide configuration and feedback rather than duplicate the gesture surface.

14.1 Status View

The status view should expose the current state: monitoring, arming, gesture capture, processing or execution. During development it can also show the most recent prediction and confidence value. During normal operation, the information can be reduced so that the application does not distract the user.

14.2 Gesture Mapping

A mapping panel can list each supported symbol and its configured command. This allows the project to demonstrate that the gesture vocabulary is not hard-coded to one application.

14.3 Configuration

The configuration interface can expose corner tolerance, activation timeout, capture timeout, model confidence threshold and command mappings. Invalid values should be detected before they are saved.

14.4 Developer Tools

A replay utility is useful during development. It can load a recorded trajectory and pass it through preprocessing and inference. This makes model experiments independent of the physical touchpad and provides a reproducible debugging workflow.

15. Testing Strategy

Testing should be performed at four levels: unit, integration, replay and physical user testing. Unit tests should cover deterministic functions such as coordinate normalization, corner classification, timeout handling and feature extraction.

15.1 State-Machine Testing

Every state transition should have at least one test. The test suite should include correct activation, incorrect corner order, activation timeout, gesture timeout, empty trajectory, successful recognition, rejected recognition and command failure.

15.2 ML Regression Testing

A small collection of representative trajectories should be retained as regression data. Whenever preprocessing or the model changes, these trajectories can be evaluated again to detect unexpected degradation.

15.3 Command Testing

Command implementations should be tested using mocks or dry-run modes where possible. Unit tests should verify that unknown identifiers are rejected and that disabled commands cannot execute.

15.4 End-to-End Testing

The final physical test should begin from normal monitoring and follow the complete interaction: corner activation, drawing, classification, validation and command execution. The same scenario should be repeated across multiple users and drawing styles.

16. Technology Stack and Resources

Layer	Technology / Resource	Purpose
Application	Java 17 or course-approved JDK	Primary implementation
Desktop UI	JavaFX	Configuration and status
Build	Maven or Gradle	Dependencies and packaging
Testing	JUnit 5	Automated tests
Logging	SLF4J + Logback	Diagnostics
Data preparation	Python + permitted ML tools	Training experiments
Inference	ONNX Runtime Java or equivalent	Local model execution
Version control	Git / GitHub	Team collaboration
Hardware	Laptop touchpad	Physical gesture input
Packaging	jpackage or approved equivalent	Desktop distribution

The training environment and deployment environment should be documented separately. If Python is used to experiment with models, the final report should state that the deployed application remains Java-based and explain how the trained model is transferred into the Java inference environment.

Version information matters for reproducibility. The project documentation should record the Java version, build tool version, ML runtime version, model version, dataset version and operating-system environment used for the final demonstration.

16.1 Repository Organization

The repository should contain source code, configuration templates, tests, documentation and small replay examples. Large datasets and generated model artifacts should not be committed casually. A clear repository structure also allows each member to work on a separate subsystem without creating an unmaintainable monolithic source file.

17. Development Workflow

The recommended workflow is incremental. The team should first establish hardware feasibility, then implement activation, then record trajectories, then construct the dataset and only afterward finalize the model. This sequence prevents the common mistake of developing a classifier before knowing whether the deployed input source can provide compatible data.

17.1 Phase I — Feasibility

Build the smallest possible touchpad diagnostic program. Record coordinate samples and verify that the operating system and Java integration provide the required information. Do not add ML or command execution at this stage.

17.2 Phase II — Interaction

Implement the four-corner state machine and trajectory recorder. At this point the team should be able to activate the system and save a gesture to disk without classification.

17.3 Phase III — Dataset and ML

Collect labeled trajectories, implement preprocessing and establish a baseline classifier. Evaluate errors and refine the feature representation before selecting the final model.

17.4 Phase IV — Java Integration

Integrate the trained model into the Java application, connect prediction results to the command registry and build the configuration interface.

17.5 Phase V — Validation

Run physical tests, replay tests and user-independent evaluation. Record limitations rather than hiding failures. The final documentation should contain enough information for another student to understand how the system was built.

18. Risks and Limitations

The largest technical risk is access to low-level touchpad input. Laptop hardware and operating systems differ, and a standard Java desktop API may not expose all the information required by the concept. The project should therefore treat the input adapter as a replaceable subsystem.

The second major risk is dataset quality. If the training data contains only one person's drawings or lacks meaningful variation, measured accuracy may not reflect real usage. This risk is mitigated by collecting data from multiple users and reserving independent test samples.

Another limitation is gesture ambiguity. Two symbols may have similar trajectories, especially when drawn quickly or at small sizes. The project should avoid choosing a command vocabulary that is visually difficult to distinguish merely for the sake of having more classes.

Command execution introduces a separate reliability requirement. A prediction error can have a visible consequence even when the classifier's numerical performance appears acceptable. Confidence thresholds, allowlisting and safe failure behavior are therefore part of the core system rather than optional security features.

18.1 Scope Limitations

The project is not intended to become a general handwriting-recognition platform. It is a compact command interface with a predefined vocabulary. It does not require cloud services, unrestricted shell execution, privileged operations or universal language recognition. Keeping these boundaries explicit makes the semester project technically achievable and easier to evaluate.

19. Team Responsibilities

Team Member	Primary Responsibility	Expected Deliverables
Ashwath Raj Mulakalapalli	Architecture and Java integration	Application structure, interfaces, FSM integration, system integration
BARATAM PRANEETH GUPTA	Machine learning and dataset	Data collection, preprocessing, training, evaluation
Harsha Vardhan Kumar	Touchpad input subsystem	Input acquisition, corner detection, trajectory capture
Sai thanush reddy	UI, testing and documentation	JavaFX interface, test harness, documentation and demo support

The allocation separates primary responsibilities while retaining shared understanding. Each member should be capable of explaining the entire pipeline during the final evaluation. Module boundaries should be agreed before implementation so that work can proceed in parallel.

19.1 Collaboration and Version Control

The repository should use feature branches or clearly separated commits for major modules. Commit messages should describe actual engineering changes. Configuration and interface definitions should be agreed before multiple modules depend on them. The team should also maintain a short development log describing dataset revisions, model experiments and significant architecture decisions.

A shared replay dataset is especially valuable for collaboration. Once a trajectory is recorded and labeled, every team member can test their module against the same input without requiring access to the physical laptop.

20. Expected Experimental Results and Reporting

The final report should present measured results rather than invented target numbers. Because the dataset and physical input quality are implementation-dependent, this document does not assign a fabricated accuracy, latency or activation success rate. Those values should be inserted after the team completes the experiments.

20.1 Recommended Result Tables

The final evaluation can report the number of classes, total trajectories, number of contributors, training/validation/test split, selected model, accuracy, macro-averaged precision, macro-averaged recall, macro-averaged F1-score, rejection rate and average inference latency.

A second experiment can measure physical interaction reliability: successful four-corner activations divided by attempts, incomplete gesture rate, rejected gesture rate, average time from activation to prediction and successful command execution rate.

20.2 Interpretation

A strong result is not defined by one metric alone. For a command interface, the relationship between false recognition and safe rejection matters. A model that occasionally rejects an ambiguous gesture may be more operationally appropriate than a model that attempts to classify every ambiguous trajectory and consequently produces incorrect commands.

20.3 Reproducibility

Every reported metric should be associated with the dataset version, preprocessing configuration and model version used to obtain it. If a model is retrained, the previous result should remain traceable. This makes the final report an engineering record rather than only a description of the finished interface.

21. Discussion

GLYVEX demonstrates how a small machine-learning model can be embedded inside a larger interactive system. The machine-learning component is important, but it is not the sole source of complexity. Hardware input, interaction segmentation, preprocessing, uncertainty and command policy all influence the quality of the final experience.

The four-corner activation mechanism is particularly useful because it establishes an explicit context boundary. Without such a boundary, every touchpad movement would need to be considered a potential gesture, creating a conflict with ordinary pointing behavior. The activation sequence allows the application to remain passive until the user deliberately requests gesture mode.

The separation between prediction and command execution also creates a useful engineering boundary. The classifier can be improved independently of the command layer, and the command layer can be tested using synthetic predictions. This separation reduces coupling and makes the system easier to reason about.

The project also illustrates a broader principle of applied ML: data representation can be as important as model selection. If the trajectory is poorly normalized, a more sophisticated classifier may simply learn irrelevant properties such as location or drawing scale. Conversely, a compact representation can make a simple model effective.

21.1 Practical Interpretation

The project should be considered a prototype of an alternative desktop interaction technique rather than a replacement for existing input methods. Its value lies in demonstrating the complete sensing-to-action pipeline and in evaluating whether a small vocabulary of learned gestures can operate reliably on a real touchpad.

22. Future Extensions

Several extensions are possible after the core system is stable. The gesture vocabulary could become user-configurable, allowing individuals to associate their own symbols with commands. A calibration stage could estimate the most reliable corner tolerance for a particular touchpad.

A more advanced model could incorporate the temporal sequence directly instead of reducing the trajectory to manually engineered features. This could allow the system to distinguish gestures that have similar geometry but different drawing order or movement dynamics.

Personalized models are another possible direction. A general model could provide initial recognition, after which user-specific examples could be collected to adapt the decision boundaries. Such adaptation would need careful evaluation to ensure that personalization improves performance without making the model brittle.

The interface could also support multimodal confirmation. For example, an uncertain prediction could require a small secondary action before execution. This would shift the system from an automatic command interface toward a confidence-aware interaction model.

Finally, the input abstraction could support additional devices such as external touch surfaces or stylus-like sensors. Because the rest of the application consumes normalized trajectory data, such extensions would primarily affect the input adapter.

23. Conclusion

GLYVEX presents a Java-based approach to touchpad gesture interaction in which a deliberate four-corner sequence activates a temporary drawing mode, a machine-learning model recognizes a predefined symbol and a controlled command layer converts the prediction into a desktop action. The project combines touch input integration, finite-state interaction design, trajectory preprocessing, supervised classification and Java application engineering.

The most important architectural decision is the separation of responsibilities. The input layer acquires physical data; the state machine decides when that data constitutes an interaction; preprocessing makes trajectories comparable; the ML model predicts a gesture class; and the command registry determines whether that class is allowed to trigger an action. This separation makes the system testable and limits the consequences of ML uncertainty.

The project should be evaluated through real measurements rather than claims of performance. Dataset diversity, user-independent testing, confusion analysis, interaction reliability and command safety should all be included in the final experimental results. The physical touchpad interface should also be treated as a first-class engineering problem rather than assumed to work automatically through standard Java GUI events.

Within a semester-project scope, a compact and reliable gesture vocabulary is preferable to an unnecessarily broad system. The resulting prototype can demonstrate a complete chain from physical input to machine interpretation to controlled software action while providing clear opportunities for future research in personalized gesture recognition and alternative desktop interaction.

References and Technical Resources

[1] Oracle. *Java Platform, Standard Edition Documentation*. Java language and runtime documentation.

[2] OpenJFX. *JavaFX Documentation*. Desktop graphical user-interface development for Java.

[3] JUnit Team. *JUnit 5 User Guide*. Unit and integration testing for Java applications.

[4] Eclipse Foundation. *Eclipse Collections and Java Ecosystem Documentation*. Supporting Java application development resources.

[5] ONNX Runtime. *ONNX Runtime Documentation*. Local machine-learning model inference and deployment resources.

[6] Scikit-learn Developers. *scikit-learn User Guide*. Classical machine-learning algorithms, preprocessing and evaluation methodology.

[7] Bishop, C. M. *Pattern Recognition and Machine Learning*. Springer. Background on supervised learning and statistical pattern recognition.

[8] Wobbrock, J. O., Wilson, A. D., and Li, Y. “Gestures without Libraries, Toolkits or Training: A $1 Recognizer for User Interface Prototypes.” Proceedings of the ACM Symposium on User Interface Software and Technology.

[9] Rubine, D. “Specifying Gestures by Example.” Proceedings of the ACM SIGGRAPH Computer Graphics conference.

[10] Git Documentation. *Git Reference Manual*. Distributed version control and collaborative development.

Final submission note

Before submission, replace the experimental placeholders with the team's actual measured dataset size, model metrics, latency measurements and hardware/input implementation details. Do not insert estimated values as if they were measured results.

Appendix A — Proposed Configuration

A configuration file can keep interaction parameters separate from source code. The exact format may be JSON, YAML or Java properties depending on the team's implementation choice.

Parameter	Meaning	Example role
cornerTolerance	Allowed distance from each corner	Activation reliability
cornerTimeoutMs	Maximum interval between corner taps	Sequence reset
debounceMs	Minimum interval between duplicate events	Input stability
gestureTimeoutMs	Maximum gesture capture duration	Capture reset
confidenceThreshold	Minimum accepted prediction score	Safe rejection
modelPath	Local model artifact	Inference
gestureMapping	Class-to-command association	Execution policy

The application should validate every configuration value at startup. Negative durations, invalid thresholds, missing model files and unknown command identifiers should cause a clear configuration error rather than an obscure runtime failure.

A.1 Example Semantic Mapping

A command mapping should use semantic identifiers rather than executable strings. For example, GESTURE_C may map to OPEN_CALCULATOR, while GESTURE_SQUARE may map to OPEN_EDITOR. The command implementation remains responsible for deciding how the application is opened.

Appendix B — Suggested Project Directory

A clean directory structure makes the project easier to build, test and present.

src/main/java/com/glyvex/app — application lifecycle and dependency wiring.

src/main/java/com/glyvex/input — touchpad event adapters and normalized samples.

src/main/java/com/glyvex/gesture — activation sequence, state machine and trajectory capture.

src/main/java/com/glyvex/processing — normalization, resampling and feature extraction.

src/main/java/com/glyvex/ml — model loading, prediction and confidence handling.

src/main/java/com/glyvex/commands — allowlisted command implementations.

src/main/java/com/glyvex/config — configuration loading and validation.

src/main/java/com/glyvex/ui — JavaFX views and controllers.

src/test/java/com/glyvex — unit, replay and integration tests.

data/ — local development datasets or references, excluded from the repository when appropriate.

models/ — versioned model artifacts with metadata.

docs/ — experiment logs, diagrams if later required, test records and final documentation.

Implementation priority

Input feasibility → four-corner state machine → trajectory recording → dataset → preprocessing → baseline model → evaluation → Java inference → command registry → UI → end-to-end testing.